
EMBARCADERO CONFERENCE
PATROCINADOR MASTER:
embarcadero
An Idera, Inc. Company

SPEC-DRIVEN DEVELOPMENT NO DIA A DIA COM DELPHI

ADRIANO SANTOS

Como sair do prompt improvisado para um método: os cinco arquivos de especificação que guiam o agente de IA — aplicados na modernização de um sistema Delphi legado, do formulário ao teste.

AVALIE
a palestra

O PONTO DE PARTIDA

VOCÊ JÁ USA IA. E MESMO ASSIM...

- **O código funciona isolado, mas não encaixa no projeto.**
- Nomes fora do padrão. Camada pulada. Componente que o time abandonou.
- A mesma decisão é refeita a cada nova sessão do agente.
- Você revisa mais do que economiza — e desconfia do resultado.

Não é falta de capacidade do agente. **É ausência de especificação.**

AVALIE
a palestra

DIAGNÓSTICO

UM PROMPT VAGO NÃO É UM PEDIDO. É UM SORTEIO.

O QUE VOCÊ DIGITA

"Cria uma função pra validar o CPF do cliente e salvar no banco."

O QUE O AGENTE DECIDE SOZINHO

- Em qual unit isso vai morar?
- Regra no formulário, no Service ou no banco?
- FireDAC direto ou passando por Repository?
- Devolve Boolean, lança exceção ou grava log?
- CPF é único por filial ou global?
- Precisa de transação? Precisa de teste?

Sete decisões não especificadas = sete palpites. E palpite não versiona.

AVALIE
a palestra

O MÉTODO

SPEC-DRIVEN DEVELOPMENT

A especificação vira o artefato primário. O código passa a ser projeção dela.

01
CONSTITUTION
princípios invioláveis
HUMANO

02
SPECIFY
o quê e por quê
HUMANO

03
PLAN
o como técnico
HUMANO + IA

04
TASKS
unidades executáveis
IA

05
IMPLEMENT
execução e revisão
IA

A participação humana cai da esquerda para a direita. A velocidade sobe.

AVALIE
a palestra

ONDE ISSO MORA

OS CINCO ARQUIVOS DENTRO DO SEU PROJETO DELPHI

```
SistemaComercial\
├── docs\
│   └── spec\
│       ├── CONSTITUTION.md   ← regras invioláveis
│       ├── SPECIFY.md        ← o quê e por quê
│       ├── PLAN.md           ← o como técnico
│       └── tasks\
│           ├── T-010.md
│           ├── T-011.md
│           └── T-012.md   ← executada pela IA
├── src\
│   ├── uCadClienteForm.pas
│   └── uCliente.Service.pas
├── tests\
│   └── uCliente.Service.Test.pas
└── SistemaComercial.dproj
```

REGRAS DA CASA

- Versionado no Git, ao lado do .dproj.
- Alteração de spec entra por Pull Request.
- Specs vêm antes do primeiro commit da feature.
- Toda regra e toda task têm identificador.

Especificação não é anexo do projeto. É parte do repositório.

AVALIE
a palestra

ARTEFATO 01 DE 05

CONSTITUTION — A CARTA MAGNA DO PROJETO

As regras que a IA não pode violar — em nenhuma task, em nenhuma sessão, por nenhum motivo.

AR-
ARQUITETURA
Camadas, separação de responsabilidade, limites

QC-
QUALIDADE
Nomenclatura, tamanho de método, cobertura

ST-
STACK
Versão do Delphi, bibliotecas aprovadas

CP-
COMPATIBILIDADE
SO suportado, banco, política de breaking change

GV-
GOVERNANÇA
Quem aprova, política de branch e de commit

SE-
SEGURANÇA
LGPD, SQL injection, dado sensível em log

Rara e estável: muda em meses, não em sprints. Comece com 12 regras, não com 80.

AVALIE
a palestra

ARTEFATO 01 · PROMPT DE CRIAÇÃO

GERANDO A CONSTITUTION DO SEU SISTEMA

```
Você é um arquiteto Delphi sênior. Vamos escrever o CONSTITUTION.md
do projeto Sistema Comercial (Delphi 12 Athens, VCL, FireDAC, Firebird).

Analise o código em src\ e proponha regras em 6 blocos:
Arquitetura, Qualidade, Stack, Compatibilidade, Governança e Segurança.

Requisitos:
- Cada regra recebe identificador (AR-01, QC-01, ST-01, SE-01...)
- Regras concretas e verificáveis, nunca aspiracionais
- Máximo de 20 regras nesta primeira versão
- Se um padrão atual do código for ruim, aponte e proponha o correto
- Não escreva nenhuma linha de implementação nesta etapa

Antes de escrever, me faça as perguntas que ainda faltam.
```

POR QUE FUNCIONA

- O identificador é o que torna a regra citável no commit.
- “Verificável, não aspiracional” corta texto motivacional.
- A última linha é a mais valiosa: obriga a IA a perguntar em vez de inventar.

Rodou em 15 minutos o que o time levaria meio dia de reunião para escrever.

AVALIE
a palestra

ARTEFATO 01 · RESULTADO

CONSTITUTION.MD

```
# CONSTITUTION — Sistema Comercial
Versão: 1.0 | Tech Lead: Adriano Santos

## Bloco 1 — Arquitetura
AR-01: Camadas obrigatórias: UI (VCL), Service, Repository, Model.
AR-02: Formulário nunca acessa TFDQuery — somente via Repository.
AR-03: Regra de negócio vive em Service. Nunca em evento de componente.
AR-04: Dependência declarada por interface, nunca por classe concreta.

## Bloco 2 — Qualidade de Código
QC-01: Método com no máximo 30 linhas.
QC-02: Toda classe de Service tem teste DUnitX correspondente.
QC-03: Prefixos obrigatórios: T classe, I interface, E exceção, F campo.

## Bloco 3 — Stack
ST-01: Delphi 12 Athens. Dados via FireDAC. Testes com DUnitX.
ST-02: Proibido TADOQuery e qualquer componente descontinuado.

## Bloco 6 — Segurança
SE-01: SQL sempre parametrizado. Concatenar SQL é proibido.
SE-02: Nenhum dado pessoal em log, ShowMessage ou mensagem de erro.
```

Um commit que diz “ajuste no cadastro (AR-03)” é rastreável. “Ajuste no cadastro” não é.

AVALIE
a palestra

ARTEFATO 02 DE 05

SPECIFY — O QUÊ E POR QUÊ, SEM UMA LINHA DE TECNOLOGIA

01
CONTEXTO
O domínio e o problema real

02
PERSONAS
Quem usa e quem é afetado

03
USER STORIES
Como... quero... para...

04
REGRAS
Invariantes com RN-001, RN-002

05
FORA DE ESCOPO
O que a feature não faz

TESTE DE PUREZA Citou FireDAC, unit ou TForm? Não é Specify — é Plan.

O documento que um gerente comercial consegue ler, entender e aprovar.

AVALIE
a palestra

ARTEFATO 02 · PROMPT DE CRIAÇÃO

EXTRAINDO A REGRA DE DENTRO DO CÓDIGO LEGADO

```
Vamos escrever a SPECIFY.md da feature Cadastro de Cliente.

Contexto: hoje a regra está dentro do evento btnSalvarClick da unit
uCadClienteForm.pas. Vou colar o código legado abaixo.

Sua tarefa:
1. Extrair do código as regras de negócio que estão implícitas
2. Escrever cada regra com identificador (RN-001, RN-002...)
3. Escrever as user stories no formato Como / Quero / Para
4. Escrever os critérios de aceite em Dado / Quando / Então
5. Listar o que NÃO faz parte desta feature

Proibido nesta etapa: citar Delphi, FireDAC, nome de unit, nome de
classe ou qualquer decisão de implementação. Se não conseguir descrever
uma regra sem citar tecnologia, me pergunte em vez de decidir.

<cole aqui o código do btnSalvarClick>
```

O GANHO REAL

- A IA faz a arqueologia do legado por você.
- Regra implícita em IF vira regra explícita com número.
- O que estava só na cabeça do dev antigo vira documento.

Em sistema legado, este é o prompt que mais devolve valor no primeiro dia.

AVALIE
a palestra

ARTEFATO 02 · RESULTADO

SPECIFY.MD

```
# SPECIFY — Cadastro de Cliente
Versão: 1.0 | Status: aprovado pelo time comercial

## Contexto
O vendedor cadastra o cliente durante o atendimento. Hoje é possível
gravar o mesmo CPF duas vezes na mesma filial, o que duplica o
histórico de crédito e trava o faturamento no fechamento do mês.

## Regras de Negócio
RN-001: CPF é único por filial — não global. Matriz e filial podem
        atender o mesmo cliente de forma independente.
RN-002: Pessoa jurídica exige CNPJ válido; física exige CPF válido.
RN-003: Cliente sem compra há mais de 24 meses exige liberação do
        gerente antes de novo pedido.

## Critérios de Aceite
Dado um CPF já cadastrado na mesma filial
Quando o vendedor tentar gravar o cliente
Então o sistema recusa e mostra qual cliente já existe (RN-001)

## Fora de escopo
Consulta a birô de crédito. Cadastro de estrangeiro sem CPF.
```

Repare: nenhuma palavra sobre Delphi, banco ou tela. De propósito.

AVALIE
a palestra

O ERRO NÚMERO 1

A FRONTEIRA ENTRE NEGÓCIO E ENGENHARIA

SPECIFY · o quê e por quê

- “CPF é único por filial” (RN-001)
- “Saber qual cliente já existe”
- “Cliente parado há 24 meses exige liberação”

PLAN · o como técnico

- UNIQUE (CPF, FILIAL_ID) na tabela CLIENTE
- EClienteDuplicado carrega o Id existente
- DataUltimaCompra indexado; regra no Service

REGRA DE OURO Exige entender o negócio para decidir? É Specify. Dá para decidir sem isso? É Plan.

AVALIE
a palestra

ARTEFATO 03 DE 05

PLAN — O QUE O SÊNIOR DECIDIRIA ANTES DO PRIMEIRO COMMIT

01
STACK DETALHADA
Versões exatas, componentes e configuração desta feature

02
ARQUITETURA DE UNITS
Quais units nascem, quais mudam e o papel de cada uma

03
MODELO DE DADOS
Tabelas, colunas, índices e script de migração

04
CONTRATOS
Interfaces com assinatura completa dos métodos

05
ESTRATÉGIA DE TESTES
Cenários, mocks e cobertura esperada

06
RISCOS
O que pode quebrar no legado e como mitigar

O Plan não repete a Specify — ele traduz a Specify para o vocabulário de engenharia.

AVALIE
a palestra

ARTEFATO 03 · PROMPT DE CRIAÇÃO

TRADUZINDO NEGÓCIO EM ENGENHARIA

```
Com base no CONSTITUTION.md e na SPECIFY.md em anexo, escreva o PLAN.md.

Precisa conter:
1. Quais units serão criadas ou alteradas, com a responsabilidade de cada
2. As interfaces (contratos) com assinatura completa dos métodos
3. Modelo de dados: tabela, colunas, índices e script de migração
4. Tratamento de erro: quais exceções e qual hierarquia entre elas
5. Estratégia de testes DUnitX: cenários e mocks necessários
6. Riscos da alteração no legado e como mitigar cada um

Respeite obrigatoriamente as regras AR-01 a AR-04, QC-02 e ST-01.
Onde a Specify estiver ambígua, liste a ambiguidade em vez de decidir
sozinho. Não gere implementação ainda — apenas o plano.
```

A LINHA QUE SALVA

- “Liste a ambiguidade em vez de decidir sozinho” é o freio de mão contra alucinação.
- Citar as regras por identificador amarra o Plan à Constitution.

Revisar um plano de 2 páginas custa 10 minutos. Revisar 800 linhas de código custa o dia.

AVALIE
a palestra

ARTEFATO 03 · RESULTADO

PLAN.MD

```
# PLAN — Cadastro de Cliente

## Arquitetura de Units (AR-01)
uCliente.Model.pas        TCliente — entidade pura, sem UI e sem banco
uCliente.Repository.pas   IClienteRepository + TClienteRepositoryFD
uCliente.Service.pas      IClienteService + TClienteService (RN-001..003)
uCadClienteForm.pas       ALTERAR: remove a regra, passa a chamar o Service

## Contratos (AR-04)
IClienteService = interface
  ['{8F2A1C40-3B77-4E19-9C22-5D1E7A9B0C31}']
  function Salvar(const ACliente: TCliente): TResultadoOperacao;
end;

## Modelo de dados
ALTER TABLE CLIENTE ADD CONSTRAINT UNQ_CLIENTE_CPF_FILIAL
  UNIQUE (CPF, FILIAL_ID);   -- garante RN-001 também no banco

## Hierarquia de exceções
EClienteException = class(Exception)
  ECpfInvalido       = class(EClienteException)
  EClienteDuplicado  = class(EClienteException)  -- expõe IdExistente

## Testes DUnitX (QC-02)
TClienteServiceTest com mock de IClienteRepository — sem banco real.
Cenários: sucesso | duplicado na mesma filial | mesmo CPF em outra
filial (deve passar) | CPF inválido | cliente inativo há 24 meses
```

Este documento é o que faz a IA parar de improvisar nome de classe.

AVALIE
a palestra

ARTEFATO 04 DE 05

TASKS — A PONTE ENTRE O PLANO E A EXECUÇÃO

01
ID E TÍTULO
T-012 — o ponto de rastreabilidade

02
OBJETIVO
O que será feito, em uma única frase

03
REFERÊNCIAS
Specify e Plan que originaram a task

04
DEPENDÊNCIAS
O que precisa estar pronto antes

05
DEFINITION OF DONE
Critérios objetivos de conclusão

06
ESTIMATIVA
Máximo de 4 horas. Passou disso, quebre

Task grande demais é o erro que mais custa caro: o agente perde contexto no meio e começa a decidir sozinho.

AVALIE
a palestra

ARTEFATO 04 · PROMPT DE CRIAÇÃO

QUEBRANDO O PLANO EM UNIDADES EXECUTÁVEIS

```
A partir do PLAN.md, gere as tasks de implementação em docs\spec\tasks\.

Cada task precisa conter:
- ID, título e estimativa (máximo 4h — se passar, quebre em duas)
- Objetivo em uma frase
- Referências: qual seção da Specify e do Plan originou a task
- Dependências: os IDs que precisam estar concluídos antes
- Definition of Done: critérios verificáveis, um por linha, incluindo
  os testes DUnitX que precisam passar

Ordene por dependência e comece pela task de menor risco.
Não implemente nada ainda.
```

QUEM MANDA AQUI

- A IA gera a lista. Você corta, funde e aprova.
- Comece pelo periférico: cria momentum e revela ambiguidade antes do núcleo.

A curadoria continua humana. É aqui que a seniority aparece.

AVALIE
a palestra

ARTEFATO 04 · RESULTADO

TASKS\T-012.MD

```
## T-012 — Implementar TClienteService.Salvar
Referências: Specify RN-001, RN-002 | Plan §2, §4
Dependências: T-010 (entidade), T-011 (repository)
Estimativa: 3h

### Objetivo
Centralizar a validação de cadastro no Service, removendo a regra de
dentro do formulário.

### Definition of Done
- [ ] IClienteService e TClienteService criados (AR-03, AR-04)
- [ ] RN-001 validada antes do INSERT, com escopo por filial
- [ ] EClienteDuplicado lançada carregando o Id do cliente existente
- [ ] Nenhum acesso a TFDQuery dentro do Service (AR-02)
- [ ] Teste DUnitX: sucesso | duplicado mesma filial | mesmo CPF em
      outra filial (passa) | CPF inválido
- [ ] btnSalvarClick reduzido a chamada de Service (AR-03)
- [ ] Nenhum CPF em log ou em ShowMessage (SE-02)
- [ ] Método com no máximo 30 linhas (QC-01)
```

O CAMPO MAIS IMPORTANTE

- O DoD vale mais que o objetivo.
- O objetivo diz o que fazer. O DoD define quando “pronto” deixa de ser opinião.
- Cada item cita a regra que ele protege.

Sem DoD explícito, “terminei” é só uma sensação.

AVALIE
a palestra

ARTEFATO 05 DE 05

IMPLEMENT — O CICLO DE EXECUÇÃO

01
PICK TASK
A próxima de menor risco

02
RUN AI
Os 3 documentos no contexto

03
GENERATE
Deixe o ciclo terminar

04
REVIEW
Constitution e DoD, item a item

05
COMMIT & PR
A mensagem cita o ID da task

```
> Contexto: CONSTITUTION.md, SPECIFY.md, PLAN.md e tasks\T-012.md
> Execute a T-012 respeitando todos os artefatos carregados.
> Liste item a item do Definition of Done: o que foi atendido e o que não.
```

O sênior deixa de ser digitador e vira curador. O conhecimento técnico continua essencial — muda o ponto de aplicação.

AVALIE
a palestra

O CASO · ANTES

O LEGADO QUE TODO MUNDO TEM

```
procedure TfrmCadCliente.btnSalvarClick(Sender: TObject);
var
  Qry: TFDQuery;
begin
  Qry := TFDQuery.Create(nil);
  try
    Qry.Connection := dmPrincipal.Conn;
    Qry.SQL.Text := 'SELECT ID FROM CLIENTE WHERE CPF = ' +
                    QuotedStr(edtCPF.Text);
    Qry.Open;
    if not Qry.IsEmpty then
    begin
      ShowMessage('CPF ' + edtCPF.Text + ' já cadastrado!');
      Exit;
    end;
    // ... mais 60 linhas de validação, INSERT e mensagem
  finally
    Qry.Free;
  end;
end;
```

O QUE ESTÁ QUEBRADO

- Regra de negócio dentro do evento (AR-03)
- SQL concatenado à mão (SE-01)
- CPF exposto na tela e no log (SE-02)
- Filial ignorada — a RN-001 nem existia escrita
- Impossível testar sem abrir o formulário (QC-02)

Cinco violações em um único evento. E ninguém sabia, porque nada estava escrito.

AVALIE
a palestra

O CASO · DEPOIS

O SERVICE GERADO A PARTIR DA T-012

```
unit uCliente.Service;

type
  TClienteService = class(TInterfacedObject, IClienteService)
  private
    FRepository: IClienteRepository;   // AR-04: depende de interface
  public
    constructor Create(const ARepository: IClienteRepository);
    function Salvar(const ACliente: TCliente): TResultadoOperacao;
  end;

function TClienteService.Salvar(const ACliente: TCliente): TResultadoOperacao;
var
  IdExistente: Integer;
begin
  if not TDocumento.CpfValido(ACliente.Cpf) then            // RN-002
    raise ECpfInvalido.Create('Documento informado é inválido');

  // RN-001: unicidade por filial, nunca global
  if FRepository.TryObterIdPorCpfFilial(ACliente.Cpf,
       ACliente.FilialId, IdExistente) then
    raise EClienteDuplicado.Create(IdExistente);

  Result := FRepository.Inserir(ACliente);
end;
```

O QUE MUDOU

- 24 linhas no lugar de 70+
- Zero SQL e zero componente visual
- Testável sem abrir o Delphi
- Cada regra com o número dela no comentário

A IA não inventou esse desenho. Ela leu o Plan e executou.

AVALIE
a palestra

O CASO · A PROVA

O TESTE QUE NASCE JUNTO E O FORMULÁRIO QUE SOBRA

```
[Test]
procedure TClienteServiceTest.NaoGravaCpfDuplicadoNaMesmaFilial;
begin
  FRepoMock.Registrar('11144477735', 1, 99);
  Assert.WillRaise(
    procedure
    begin
      FService.Salvar(ClienteValido('11144477735', 1));
    end,
    EClienteDuplicado);
end;

[Test]                                                  // RN-001
procedure TClienteServiceTest.PermiteMesmoCpfEmOutraFilial;
begin
  FRepoMock.Registrar('11144477735', 1, 99);
  Assert.IsTrue(
    FService.Salvar(ClienteValido('11144477735', 2)).Sucesso);
end;
```

```
procedure TfrmCadCliente.btnSalvarClick(Sender: TObject);
begin
  try
    FClienteService.Salvar(MontarClienteDaTela);
    ShowMessage('Cliente salvo com sucesso.');
  except
    on E: EClienteDuplicado do
      ExibirClienteExistente(E.IdExistente);   // SE-02
  end;
end;
```

O FORMULÁRIO VIROU TELA

- 9 linhas. Sem SQL, sem regra.
- Testável sem abrir o formulário.

O DoD só é marcado quando o teste passa. E quem marca é você, não o agente.

AVALIE
a palestra

O RESULTADO INVISÍVEL

RASTREABILIDADE BIDIRECIONAL

BUG *em produção* → **COMMIT** *git log* → **T-012** *a task* → **PLAN §2** *a decisão* → **SPECIFY** *RN-001*

```
feat(cliente): valida CPF único por filial [T-012]

Implementa RN-001 conforme PLAN §2.
Remove a regra de dentro de btnSalvarClick (AR-03).
Troca SQL concatenado por query parametrizada (SE-01).
```

Em projeto legado sem SDD, entender uma regra é arqueologia. **Com SDD, é navegação.**

AVALIE
a palestra

O QUE O MÉTODO ENTREGA

MEÇA, OU VIRA RELIGIÃO

6
semanas de entrega
(previsão: 10 a 12)

88%
de cobertura de testes
sem esforço extra

3
bugs em produção/30d
(média do time: 11)

15+7
minutos por task
geração + revisão

1 dia investigando : 2 horas implementando. A proporção se inverte — e é exatamente esse o ponto.

AVALIE
a palestra

Números do estudo de caso do livro Spec-Driven Development (cap. 8), projeto real de gateway de pagamento.

ARMADILHAS

OS QUATRO ERROS QUE VÃO TE ATRAPALHAR

01
CONSTITUTION PERFEITA DE CARA
Vira documento morto e o time abandona. Comece com 12 regras e refine durante o projeto.

02
SPECIFY CONTAMINADA
Citou FireDAC ou nome de unit? Saiu da Specify e entrou no Plan. Aplique o teste de pureza.

03
TASK GRANDE DEMAIS
Acima de 4h o agente perde contexto no meio e passa a decidir por conta própria.

04
DoD MARCADO NO OLHO
O agente não marca task como concluída — você marca, depois de rodar o teste.

Bônus: não paralelize tasks no começo. Conflito de merge apaga a rastreabilidade que você acabou de construir.

AVALIE
a palestra

PLANO DE AÇÃO

O QUE FAZER NA SEGUNDA-FEIRA

- **Crie docs\spec\ no seu projeto** e versione junto com o .dproj.
- **Escreva 12 regras de Constitution em uma hora** — as óbvias que o time já segue sem escrever.
- **Pegue UMA feature pequena** e escreva a Specify sem citar Delphi uma única vez.
- **Gere Plan e Tasks com a IA** — depois corte, funda e aprove com olho de sênior.
- **Execute a menor task e meça** tempo, bugs e retrabalho. Sem número, não há argumento.

Cinco minutos de checagem antes de disparar a IA economizam horas de retrabalho depois.

AVALIE
a palestra

PARA LEVAR DAQUI

A IA NÃO SUBSTITUI O SÊNIOR. ELA MOVE O TRABALHO DO SÊNIOR DO TECLADO PARA OS OLHOS.

Quem domina a clareza antes do código ganha velocidade sem perder qualidade.

Spec-Driven Development · o livro completo, com o estudo de caso e os exercícios

AVALIE
a palestra

EMBARCADERO CONFERENCE 26

PATROCINADOR MASTER: embarcadero® An Idera, Inc. Company

QUER ME VER NA #ECON27?
ACESSE O QRCODE E AVALIE MINHA PALESTRA!

ADRIANO SANTOS

@adrianosantostreina

in/adrianosantostreina

adriano@adrianosantostreina.com.br

adrianosantos.link/ComunidadeDevAI